Class Fundamentals:

Class:- A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

object :

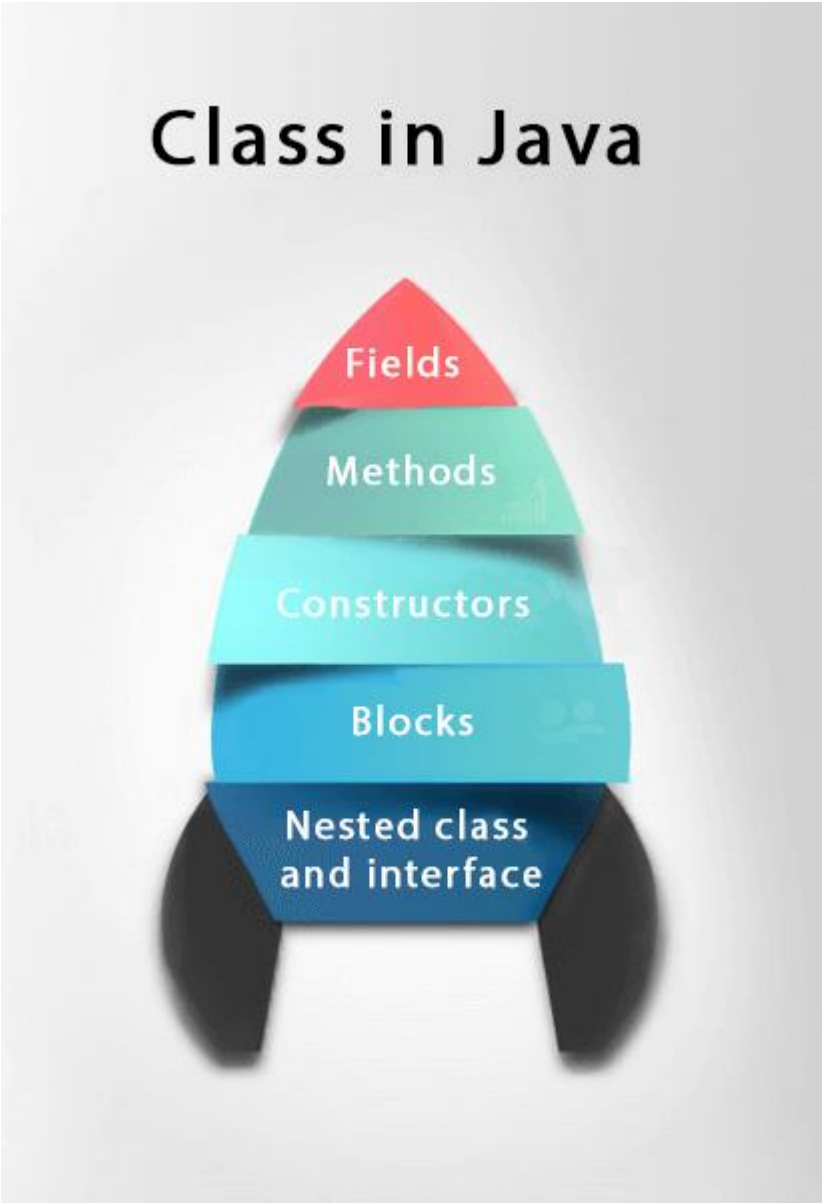
An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

An object has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

A class in Java can contain:

- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interface**



Syntax to declare a class:

```
class <class_name>{
    field;
    method;
}
```

Instance variable in Java

A variable which is created inside the class but outside the method is known as an instance variable. Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.

Method in Java

In Java, a method is like a function which is used to expose the behavior of an object.

Advantage of Method

- Code Reusability
- Code Optimization

---

new keyword in Java

The new keyword is used to allocate memory at runtime. All objects get memory in Heap memory area.

Object and Class Example: main within the class

In this example, we have created a Student class which has two data members id and name. We are creating the object of the Student class by new keyword and printing the object's value.

Here, we are creating a main() method inside the class.

File: Student.java

```
//Java Program to illustrate how to define a class and fields
//Defining a Student class.
class Student{
 //defining fields
 int id;//field or data member or instance variable
 String name;
 //creating main method inside the Student class
 public static void main(String args[]){
  //Creating an object or instance
  Student s1=new Student();//creating an object of Student
  //Printing values of the object
  System.out.println(s1.id);//accessing member through reference variable
  System.out.println(s1.name);
 }
}
```

Output:

```
0
null
```

3 Ways to initialize object

There are 3 ways to initialize object in Java.

1. By reference variable
2. By method
3. By constructor

1) Object and Class Example: Initialization through reference

Initializing an object means storing data into the object. Let's see a simple example where we are going to initialize the object through a reference variable.

File: TestStudent2.java

```
class Student{
 int id;
 String name;
}
class TestStudent2{
 public static void main(String args[]){
  Student s1=new Student();
  s1.id=101;
  s1.name="Sonoo";
 System.out.println(s1.id+" "+s1.name);//printing members with a white space
 }
}
```

Output:

```
101 Sonoo
```

2) Object and Class Example: Initialization through method

In this example, we are creating the two objects of Student class and initializing the value to these objects by invoking the insertRecord method. Here, we are displaying the state (data) of the objects by invoking the displayInformation() method.

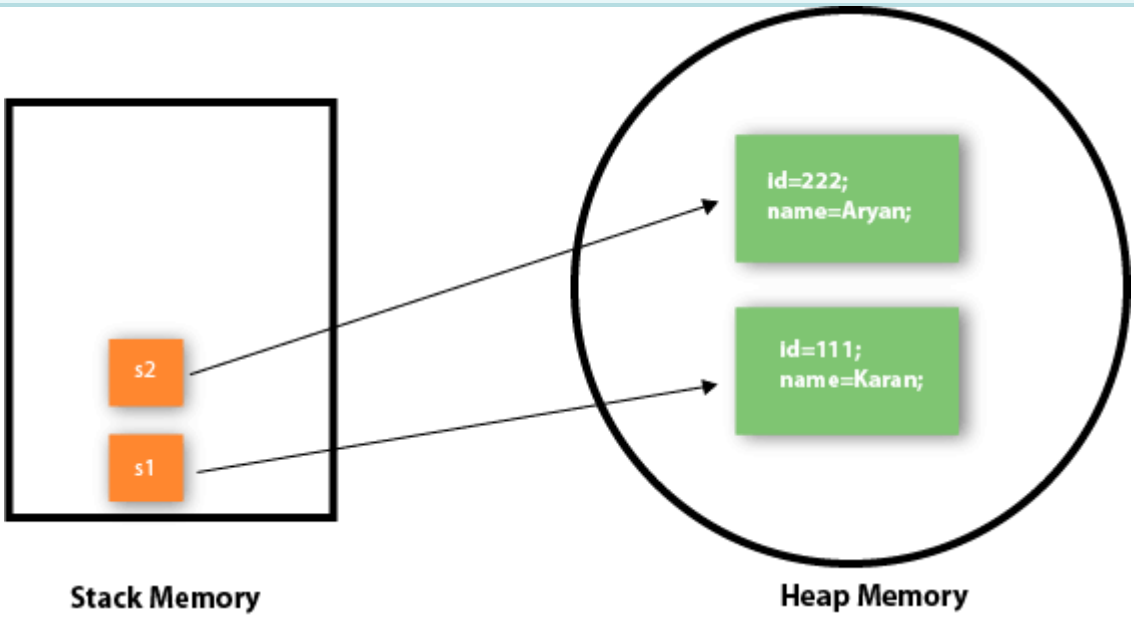
File: TestStudent4.java

```
class Student{
 int rollno;
 String name;
 void insertRecord(int r, String n){
  rollno=r;
  name=n;
 }
 void displayInformation(){System.out.println(rollno+" "+name);}
}
```

```
class TestStudent4{
 public static void main(String args[]){
  Student s1=new Student();
  Student s2=new Student();
  s1.insertRecord(111,"Karan");
  s2.insertRecord(222,"Aryan");
  s1.displayInformation();
  s2.displayInformation();
 }
}
```

Output:

```
111 Karan
222 Aryan
```



3) Object and Class Example: Initialization through a constructor

---

Object and Class Example: Employee

Let's see an example where we are maintaining records of employees.

File: TestEmployee.java

```
class Employee{
```

```
.                int id;
.                String name;
.                float salary;
.                void insert(int i, String n, float s) {
.                    id=i;
.                    name=n;
.                    salary=s;
.                }
.                void display(){System.out.println(id+" "+name+" "+salary);}
.            }
.            public class TestEmployee {
.            public static void main(String[] args) {
.                Employee e1=new Employee();
.                Employee e2=new Employee();
.                Employee e3=new Employee();
.                e1.insert(101,"ajeet",45000);
.                e2.insert(102,"irfan",25000);
.                e3.insert(103,"nakul",55000);
.                e1.display();
.                e2.display();
.                e3.display();
.            }
.            }
```

Output:

```
101 ajeet 45000.0
102 irfan 25000.0
103 nakul 55000.0
```

Reference Variable:

being an object-oriented programming language, allows the use of reference variables to work with objects and their data. In Java, objects are created dynamically on the heap memory, and reference variables are used to hold the memory address of these objects. This concept of reference variables is fundamental to Java's approach to object-oriented programming.

1) this: to refer current class instance variable

The this keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

Example:

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}

class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

```
111 ankit 5000.0
112 sumit 6000.0
```

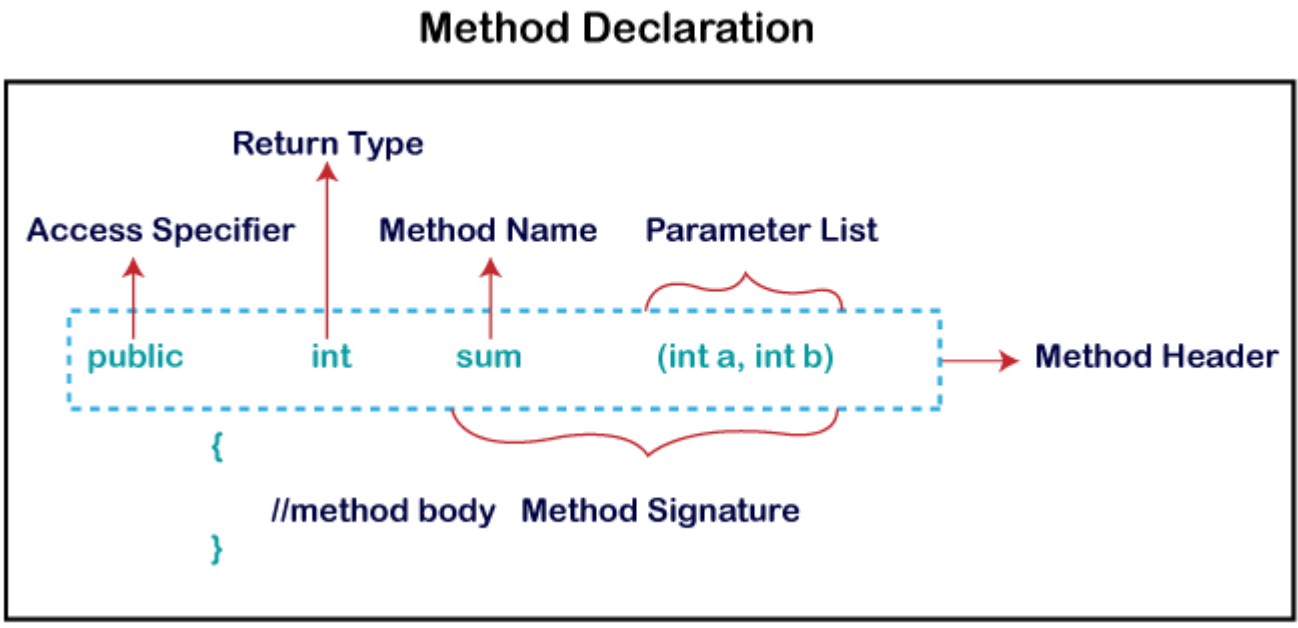
Methods :

A **method** is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation. It is used to achieve the **reusability** of code. We write a method once and use it many times. We do not require to write code again and again. It also provides the **easy modification** and **readability** of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

The most important method in Java is the **main()** method.

Method Declaration

The method declaration provides information about method attributes, such as visibility, return-type, name, and arguments. It has six components that are known as **method header**, as we have shown in the following figure.



Method Signature: Every method has a method signature. It is a part of the method declaration. It includes the **method name** and **parameter list**.

Access Specifier: Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides **four** types of access specifier:

- **Public:** The method is accessible by all classes when we use public specifier in our application.
- **Private:** When we use a private access specifier, the method is accessible only in the classes in which it is defined.
- **Protected:** When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **Default:** When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

methods can return values, and parameters can be used to pass data into methods, allowing for more flexible and reusable code. Here's an overview of how to return values from methods, use parameters, and add a parameterized method to a class (such as Vehicle).

1. Returning from a Method

A method can return a value or simply complete its execution. If a method returns a value, you need to declare the return type in the method signature (e.g., int, String, or boolean). If it doesn't return a value, it should have a void return type.

```
public int add(int a, int b) { // Method with return type int

    return a + b;

}
```

```
public void printMessage() {    // Void method (no return)

    System.out.println("Hello, World!");

}
```

- For methods with a non-void return type, use the return statement to send back a value.
- For void methods, no return statement is needed, but you can use return; to exit early.

2. Returning a Value

When you want a method to return a value, specify the return type in the method signature, then use a return statement to send back the result.

```
public class Calculator {
    public int add(int x, int y) { // Method that returns an int
        return x + y; // Returns the result of the addition
    }
}
```

In the example above, the add method takes two integers as input parameters, calculates the sum, and returns it as an int. When you call add(3, 4), it returns 7.

3. Using Parameters

Parameters allow you to pass values into methods, which can then use these values to perform operations or calculations.

Syntax:

```
public int multiply(int a, int b) { // a and b are parameters
    return a * b;
}
```

When you call multiply(5, 3), a takes the value 5 and b takes the value 3, allowing the method to return 15.

4. Adding a Parameterized Method to Vehicle

Let's say you have a Vehicle class, and you want to add a method that takes parameters to set the speed of the vehicle and return a message confirming the update. Here's an example:

```
public class Vehicle {
    private int speed;

    // Parameterized method to set speed and return a message
    public String setSpeed(int newSpeed) {
        speed = newSpeed;  // Set speed based on parameter
        return "Speed is set to " + speed + " km/h";
    }

    public int getSpeed() {  // Getter method for speed
        return speed;
    }
}
```

In this example:

- The setSpeed method is parameterized with an int newSpeed parameter.
- It assigns the value of newSpeed to the speed field of the Vehicle class and returns a confirmation message.

Constructors:

a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Types of Java Constructors

There are two types of constructors in Java:

1. Default Constructor (no-arg constructor)
2. Parameterized Constructor

Java Default Constructor

A constructor is called "Default Constructor" when it does not have any parameter.

Syntax:

```
<class_name>(){}
```

File Name: Byke1.java

```
//Java Program to create and call a default constructor
class Bike1{
//creating a default constructor
Bike1(){System.out.println("Bike is created");}
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}
}
```

Output:

```
Bike is created
```

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

File Name: Student4.java

```
//Java Program to demonstrate the use of the parameterized constructor.
class Student4{
  int id;
  String name;
  //creating a parameterized constructor
  Student4(int i,String n){
  id = i;
  name = n;
```

```
.                 }
.                 //method to display the values
.                 void display(){System.out.println(id+" "+name);}
.
.                 public static void main(String args[]){
.                 //creating objects and passing values
.                 Student4 s1 = new Student4(111,"Karan");
.                 Student4 s2 = new Student4(222,"Aryan");
.                 //calling method to display the values of object
.                 s1.display();
.                 s2.display();
.                }
.               }
```

Output:

```
111 Karan
222 Aryan
```

Garbage Collection

garbage means unreferenced objects.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.

There are many ways:

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.

1) By nulling a reference:

```
.               Employee e=new Employee();
```

```
e=null;
```

2) By assigning a reference to another:

```
Employee e1=new Employee();
Employee e2=new Employee();
e1=e2;//now the first object referred by e1 is available for garbage collection
```

3) By anonymous object:

```
new Employee();
```

finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

```
protected void finalize(){}
```

gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public static void gc(){}
```

Example:

```
public class TestGarbage1{
 public void finalize(){System.out.println("object is garbage collected");}
 public static void main(String args[]){
  TestGarbage1 s1=new TestGarbage1();
  TestGarbage1 s2=new TestGarbage1();
  s1=null;
  s2=null;
  System.gc();
 }
}
```

Output:

```
object is garbage collected
object is garbage collected
```

this:

this is a **reference variable** that refers to the current object.

Example:

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}

class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

```
111 ankit 5000.0
112 sumit 6000.0
```

Access Modifiers:

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1. **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

1) Private

The private access modifier is accessible only within the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

Example:

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Simple{
 public static void main(String args[]){
   A obj=new A();
   System.out.println(obj.data);//Compile Time Error
   obj.msg();//Compile Time Error
   }
}
```

2) Default

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
//save by A.java
package pack;
class A{
  void msg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();//Compile Time Error
```

```
    obj.msg();//Compile Time Error
  }
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifer.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.*;

class B extends A{
  public static void main(String args[]){
   B obj = new B();
   obj.msg();
  }
}
```

Output:Hello

4) Public

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
//save by A.java

package pack;
public class A{
public void msg(){System.out.println("Hello");}
}

//save by B.java

package mypack;
import pack.*;

class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

Output:Hello

Method Overloading:

Method Overloading is the feature that enables defining several methods in a class having the same name but with different parameters lists.

Example:

```
// Class Adder contains overloaded methods to add integers
class Adder {
   // Method to add two integers
   static int add(int a, int b) {
      return a + b;
   }
   // Method to add three integers
   static int add(int a, int b, int c) {
      return a + b + c;
   }
}
public class TestOverloading1 {
   public static void main(String[] args) {
      // Calling the add method with two integers
      System.out.println(Adder.add(11, 11)); // Output: 22
      // Calling the add method with three integers
      System.out.println(Adder.add(11, 11, 11)); // Output: 33
   }
}
```

Output:

```
22
33
```

Overloading Constructors:

The constructor overloading can be defined as the concept of having more than one constructor with different parameters so that every constructor can perform a different task.

Example

```
public class Student {
//instance variables of the class
int id;
String name;

Student(){
System.out.println("this a default constructor");
}

Student(int i, String n){
id = i;
name = n;
}

public static void main(String[] args) {
//object creation
Student s = new Student();
System.out.println("\nDefault Constructor values: \n");
System.out.println("Student Id : "+s.id + "\nStudent Name : "+s.name);

System.out.println("\nParameterized Constructor values: \n");
Student student = new Student(10, "David");
System.out.println("Student Id : "+student.id + "\nStudent Name : "+student.name);
}
}
```

Output:

```
this a default constructor

Default Constructor values:

Student Id : 0
Student Name : null
```

Parameterized Constructor values:

Student Id : 10
Student Name : David

Use of this () in constructor overloading

However, we can use this keyword inside the constructor, which can be used to invoke the other constructor of the same class.

```
.       public class Student {
.       //instance variables of the class
.       int id,passoutYear;
.       String name,contactNo,collegeName;
.
.       Student(String contactNo, String collegeName, int passoutYear){
.       this.contactNo = contactNo;
.       this.collegeName = collegeName;
.       this.passoutYear = passoutYear;
.       }
.
.       Student(int id, String name){
.       this("9899234455", "IIT Kanpur", 2018);
.       this.id = id;
.       this.name = name;
.       }
.
.       public static void main(String[] args) {
.       //object creation
.       Student s = new Student(101, "John");
.       System.out.println("Printing Student Information: \n");
.       System.out.println("Name: "+s.name+"\nId: "+s.id+"\nContact No.: "+s.contactNo+"\nCollege Na
me: "+s.contactNo+"\nPassing Year: "+s.passoutYear);
.       }
.       }
```

Output:

Printing Student Information:

Name: John
Id: 101
Contact No.: 9899234455
College Name: 9899234455

Passing Year: 2018

Recursion:

Recursion is a process in which a method calls itself continuously. A method in java that calls itself is called recursive method.

Syntax:

```
returntype methodname(){
//code to be executed
methodname();//calling same method
}
```

Recursion Example 1: Factorial Number

```
public class RecursionExample1 {
  static int factorial(int n){
      if (n == 1)
        return 1;
      else
        return(n * factorial(n-1));
  }

public static void main(String[] args) {
System.out.println("Factorial of 5 is: "+factorial(5));
}
}
```

Output:

Factorial of 5 is: 120

Working of above program:

factorial(5)

factorial(4)

factorial(3)

factorial(2)

factorial(1)

```
return 1
return 2*1 = 2
return 3*2 = 6
return 4*6 = 24
return 5*24 = 120
```

Recursion Example 2: Fibonacci Series

```
public class RecursionExample2 {
  static int n1=0,n2=1,n3=0;
   static void printFibo(int count){
     if(count>0){
        n3 = n1 + n2;
        n1 = n2;
        n2 = n3;
        System.out.print(" "+n3);
        printFibo(count-1);
      }
   }

public static void main(String[] args) {
  int count=15;
    System.out.print(n1+" "+n2);//printing 0 and 1
    printFibo(count-2);//n-2 because 2 numbers are already printed
}
}
```

Output:

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Inner Classes (Nested Classes):

inner class or nested class is a class that is declared inside the class or interface.

We use inner classes to logically group classes and interfaces in one place to be more readable and maintainable.

Additionally, it can access all the members of the outer class, including private data members and methods.

Syntax of Inner class

```
.        class Java_Outer_class{
.         //code
.         class Java_Inner_class{
.          //code
.         }
.        }
```

Example:

```
public class OuterClass {
    private String message = "Hello from the Outer Class!";

    // Member Inner Class
    public class InnerClass {
        public void displayMessage() {
            System.out.println(message); // Can access outer class private members
        }
    }

    public static void main(String[] args) {
        OuterClass outer = new OuterClass();
        OuterClass.InnerClass inner = outer.new InnerClass();
        inner.displayMessage();
    }
}
```

Output:
Hello from the Outer Class!

Varargs:*Varargs* (variable-length arguments) allow you to pass a variable number of arguments of the same type to a method. They are useful when you don't know how many arguments you might need to pass at runtime.

Varargs Basics

- Varargs are specified by adding an ellipsis (...) after the type in a method parameter, which allows the method to accept zero or more arguments of that type.
- Inside the method, varargs are treated like an array of that type.

Syntax:

```
public void methodName(Type... varargs) {
    // method body
}
```

Example:

```
public class VarargsExample {
    public static void printNumbers(int... numbers) {
        for (int number : numbers) {
```

```
            System.out.print(number + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printNumbers(1);            // Output: 1
        printNumbers(1, 2, 3, 4);   // Output: 1 2 3 4
        printNumbers();             // Output: (nothing)
    }
}
```

Overloading Varargs Methods

- Varargs can be combined with method overloading, but you must be careful to avoid ambiguity.
- A method can have multiple versions with different parameter types, including varargs, to allow flexible calling patterns.

Example:

```
public class VarargsOverloading {
    public static void display(String message, int... numbers) {
        System.out.println("Message: " + message);
        for (int number : numbers) {
            System.out.print(number + " ");
        }
        System.out.println();
    }

    public static void display(String message, double... numbers) {
        System.out.println("Message: " + message);
        for (double number : numbers) {
            System.out.print(number + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        display("Integers", 1, 2, 3);       // Calls the int version
        display("Doubles", 1.1, 2.2, 3.3);  // Calls the double version
    }
}
```

Output:

```
Message: Integers
1 2 3
Message: Doubles
```

1.1 2.2 3.3

Varargs and Ambiguity:

Using varargs can lead to ambiguity in method overloading when:

1. Multiple methods could be a match for the parameters.
2. The compiler can’t decide which method to use.

Example:

```
public class VarargsAmbiguity {

    public static void display(int... numbers) {

        System.out.println("Int Varargs");

    }


    public static void display(Integer... numbers) {

        System.out.println("Integer Varargs");

    }


    public static void main(String[] args) {

        display(1, 2, 3); // Error: reference to display is ambiguous

    }

}
```